

LibreSpace Boston

Purpose

Boston is expensive, and it is not always obvious where a person can exist for a few hours without spending money. Students, remote workers, recent immigrants, tourists, low-income residents, and people between appointments often need a place to sit, use Wi-Fi, cool down, or find a restroom, without buying anything. City open data answers pieces of this (Wi-Fi nodes, park polygons, park amenities, ADA accessibility, tree canopy), but only if a resident cross-references five separate government datasets themselves. LibreSpace Boston fuses those five Analyze Boston datasets into one situational assistant: a user asks a natural, multi-constraint question ("a free place near Downtown where I can sit and use Wi-Fi") and gets back a cited, practical recommendation, with any attribute the data cannot confirm (e.g. "quiet") flagged honestly instead of guessed.

Technical Architecture

A one-time Python ETL (`data-service/`) spatially joins the five datasets into a single "spots" index and generates a natural-language description per spot, embedded into a local ChromaDB collection. The Kotlin/Ktor backend loads that index into memory and serves `POST /api/query` : a structured path (location, radius, amenity filters) that never touches an LLM, and, when free-text `query` is present, a semantic path that embeds the query (a second local RamaLama instance serving `all-minilm`), re-ranks by similarity against Chroma, then runs a local RamaLama `qwen2.5:7b` chat model for language detection/translation and grounded answer synthesis with citations. There is no cloud LLM fallback, local-only by design. Postgres (Neon) backs the one piece of state that is not ETL output: anonymous per-device favorites, keyed by a client-generated device ID with no login system. A React 19/Vite webclient and an Expo (React Native) mobile app both consume a single OpenAPI-generated TypeScript client, keeping both frontends in sync with the backend.

Challenges Faced During Development

The hardest call was committing to a fully local, no-cloud-fallback LLM strategy, required by the hackathon's FOSS framing but a real capability tradeoff (a small local model is weaker at multilingual understanding than a hosted frontier model). This was resolved by verifying RamaLama could serve `qwen2.5:7b` with acceptable quality first, and by tiering language support honestly (validated English/Spanish, best-effort elsewhere with a visible disclaimer) rather than silently underperforming.

Wiring real semantic retrieval surfaced a subtler problem: the data was already embedded offline with `sentence-transformers` ' `all-MiniLM-L6-v2` (384-dim), but RamaLama serves one model per instance, and the chat model's embedding space (3584-dim) was incompatible. This required a second, separate RamaLama container serving `all-minilm` for query-time embeddings, with compatibility empirically verified via cosine similarity against vectors already sitting in Chroma, not assumed from matching model names. The local chat model also proved slow enough (tens of seconds per synthesis call) to require raising the backend's default HTTP timeout, and unreliable enough about returning `answer` as a plain JSON string (sometimes an array of fragments instead) that the parser flattens either shape defensively; both LLM calls degrade to a disclaimer on failure rather than a 500.

Future Expansion Opportunities

The natural-language query pipeline (structured filtering, semantic retrieval, grounded synthesis) is fully built and merged on the backend, but the webclient's search bar does not yet call it end to end, that wiring is next. Beyond that: multi-turn conversational follow-ups instead of single-shot Q&A, progressive-disclosure answers (summary first, expandable detail), visual/map-based answers alongside text, and PWA installability. Friends and location-sharing is explicitly deferred: the anonymous per-device favorites system it would build on is merged, but the sharing API and UI are unbuilt.